AES

Visão Geral do Advanced Encryption Standard (AES)

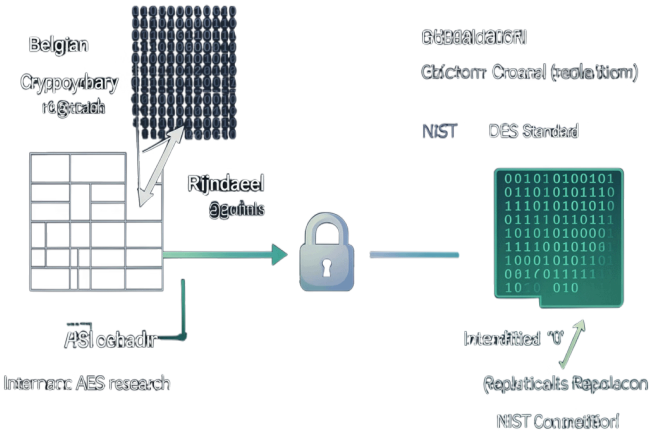
1. Sumário Executivo & Propósito

O **Advanced Encryption Standard (AES)** é o padrão internacional de criptografia simétrica de bloco (*block cipher*) estabelecido e auditado pelo *National Institute of Standards and Technology* (NIST) dos Estados Unidos (norma **FIPS PUB 197**).

Este relatório fornece uma explicação, clara e intuitiva do funcionamento do AES, detalhando sua evolução histórica, arquitetura de dados, etapas operacionais de processamento e rigor matemático subjacente.

2. Histórico e Contexto da Padronização

1. **Origem (Algoritmo Rijndael):** Desenvolvido pelos criptógrafos belgas Joan Daemen e Vincent Rijmen, o algoritmo *Rijndael* foi selecionado em outubro de 2000 como vencedor de uma competição pública internacional promovida pelo NIST para substituir o antigo padrão Data Encryption Standard (DES), que havia se tornado vulnerável a ataques de força bruta.

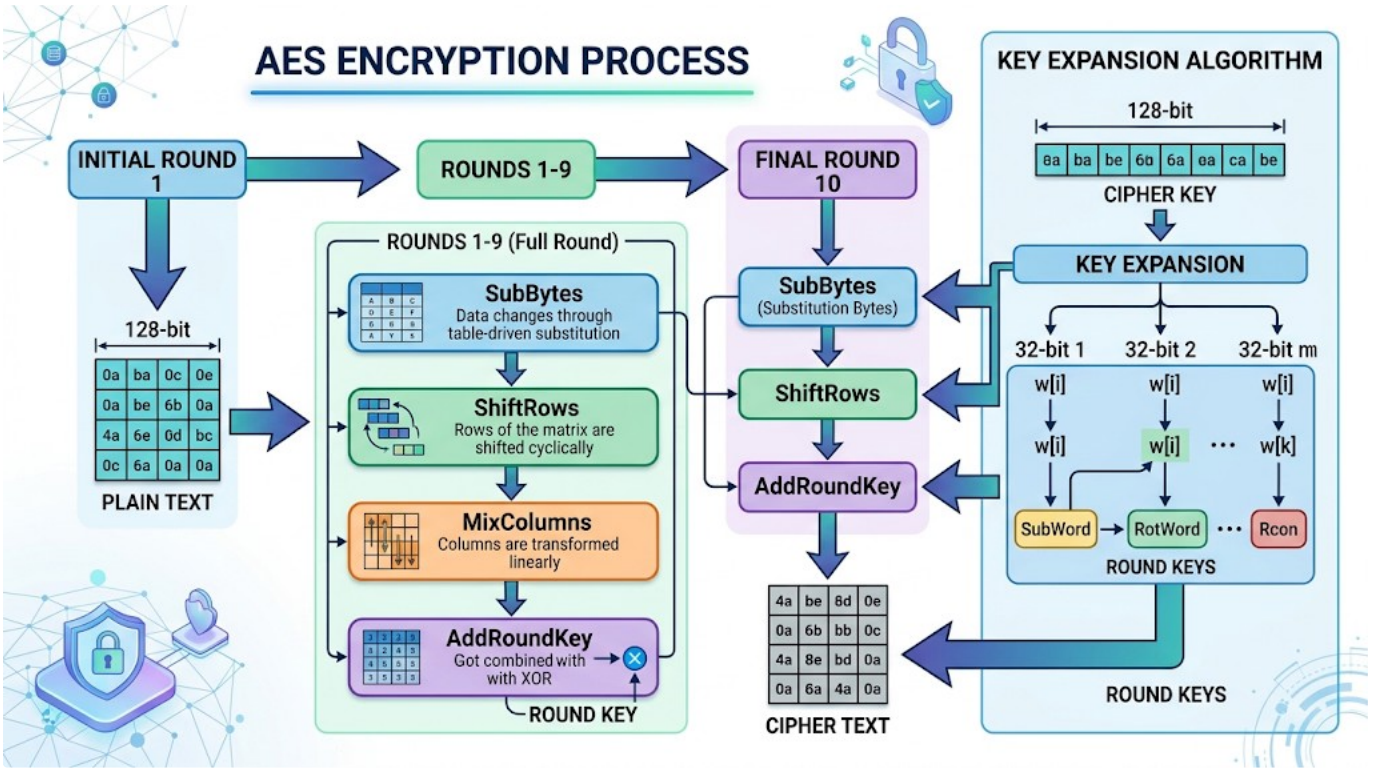


2. **Adoção e Conformidade:** É a norma oficial para a proteção de dados sigilosos e de segurança nacional do Governo Federal dos EUA, atendendo às normativas **FIPS 197** e **ISO/IEC 18033-3**. É amplamente utilizado no setor financeiro, em protocolos de comunicação segura (ex.: HTTPS/TLS, IPsec, Wi-Fi WPA2/WPA3) e no armazenamento de dados criptografados (ex.: BitLocker, FileVault).

3. Arquitetura de Dados e Variantes do Algoritmo

O AES é um algoritmo **simétrico** (a mesma chave é utilizada para criptografar e descriptografar). Ele atua sobre blocos de dados de tamanho fixo através de múltiplos ciclos de transformação (*rounds*).

O algoritmo é projetado para executar transformações de embaralhamento reversíveis — garantindo que os dados originais só possam ser restaurados por entidades em posse da chave correta.



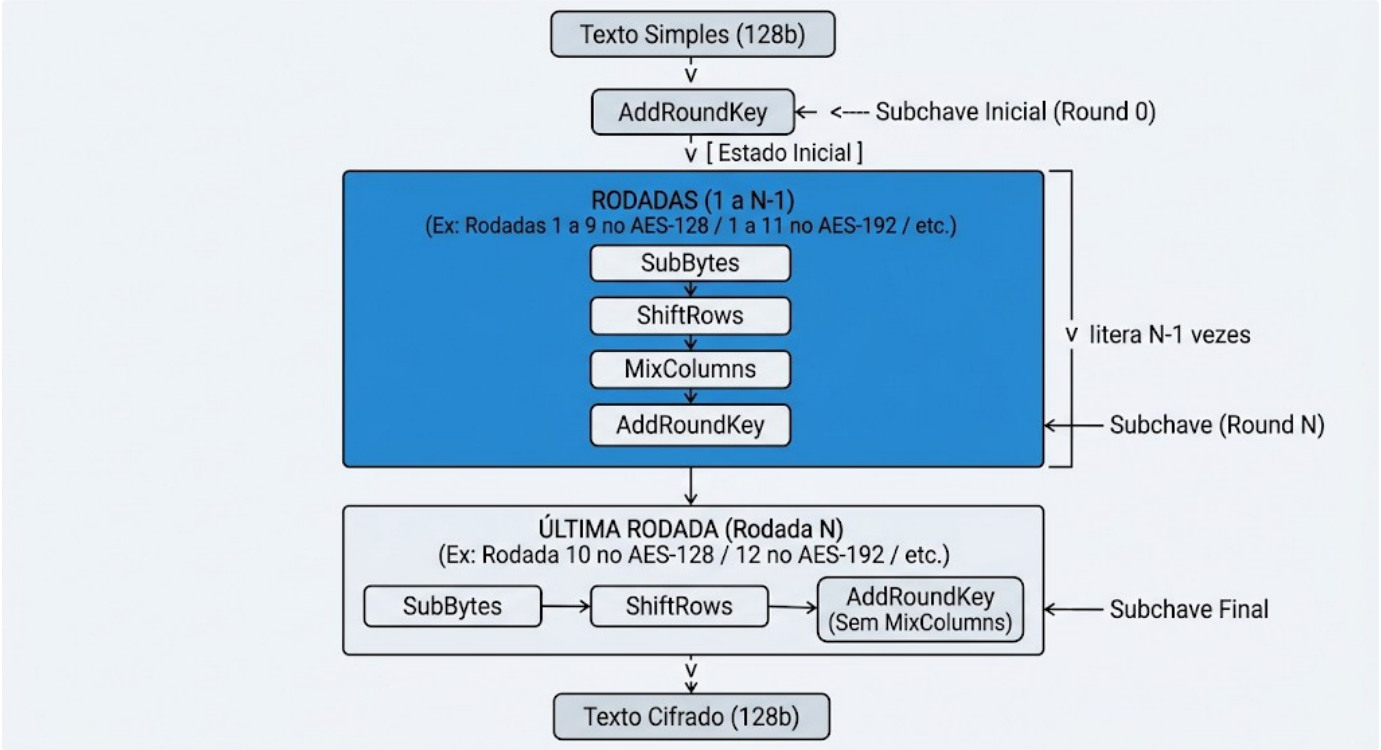
3.1. Transformações por Rodada

Durante o processo de criptografia, cada bloco de dados passa por um ciclo repetitivo composto por quatro operações principais:

1. **SubBytes**
2. **ShiftRows**
3. **MixColumns**
4. **AddRoundKey**

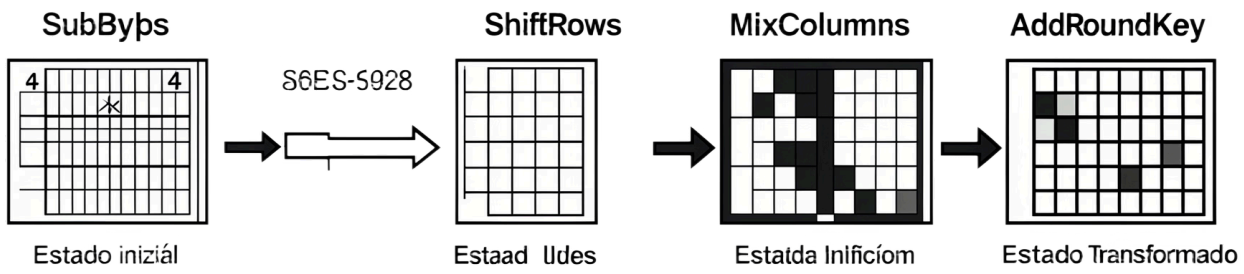
3.2. Funcionamento Interno do Processamento

Abaixo está o diagrama do fluxo completo do algoritmo, demonstrando como uma mensagem em texto simples (*plain text*) é transformada no texto cifrado (*cipher text*).



3.2.1. As Quatro Transformações Principais por Rodada

Em cada ciclo (*round* completo), a matriz de **Estado** de 128 bits passa sequencialmente por quatro etapas de manipulação matemática:



3.2. SubBytes e ShiftRows reorganizam o Estado

1. SubBytes (Substituição de Bytes)

- **O que faz:** Substituição não linear byte a byte utilizando uma tabela fixa chamada *S-Box*.
- **Objetivo:** Adicionar **confusão** ao processo, eliminando padrões diretos entre o texto original e o texto cifrado.

S-Box

Substituição não linear

2. ShiftRows (Deslocamento de Linhas)

- **O que faz:** Permutação onde as linhas da matriz 4 × 4 são deslocadas ciclicamente para a esquerda por diferentes posições (a 1ª linha não desloca, a 2ª desloca 1 byte, a 3ª desloca 2 bytes e a 4ª desloca 3 bytes).
- Internamente, esses 16 bytes são dispostos em uma matriz de 4 × 4 bytes denominada **Estado** (*State Matrix*), conforme ilustrado abaixo:
- **Objetivo:** Garantir a difusão dos bytes entre as colunas do bloco.

b_0	b_4	b_8	b_{12}
b_1	b_5	b_9	b_{13}
b_2	b_6	b_{10}	b_{14}
b_3	b_7	b_{11}	b_{15}

3.2. MixColumns e AddRoundKey completam a rodada

3. MixColumns (Mistura de Colunas)

- **O que faz:** Operação de multiplicação matricial onde cada coluna da matriz de Estado é combinada matematicamente.
- *Nota: Esta etapa é omitida na última rodada.*
- **Objetivo:** Garantir alta **difusão**, de modo que a alteração de um único bit no texto simples altere quase metade dos bits do texto cifrado (efeito avalanche).

4. AddRoundKey (Adição da Chave da Rodada)

- **O que faz:** Realiza uma operação bit a bit de OU Exclusivo (**XOR**) entre a matriz de Estado atual e a subchave correspondente àquela rodada específica.
- **Objetivo:** Conectar diretamente os dados à chave criptográfica secreta.

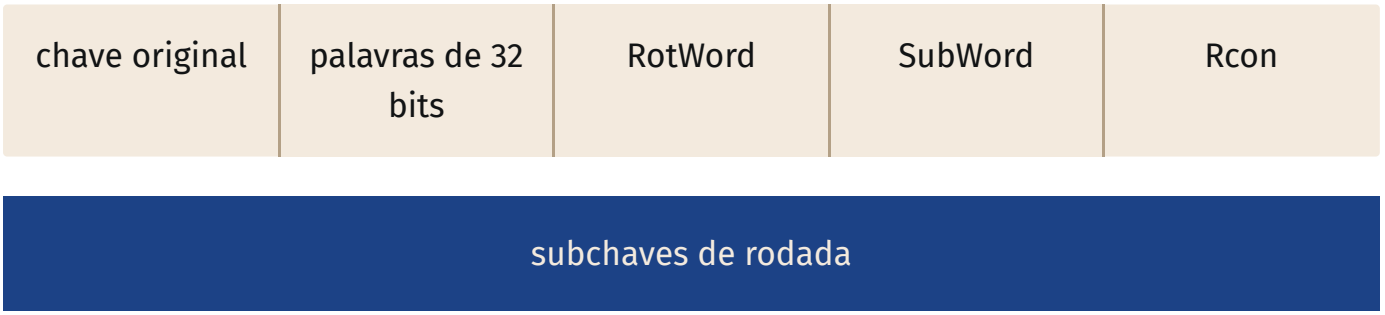
4. Expansão de Chave (Key Expansion)

A chave primária fornecida é processada por uma rotina técnica denominada **KEYEXPANSION()**. Essa função gera subchaves exclusivas que alimentam a etapa de **AddRoundKey** em cada ciclo do algoritmo.

A rotina **KeyExpansion()** do algoritmo AES é responsável por transformar a chave primária fornecida pelo usuário (de 128, 192 ou 256 bits) em um conjunto estendido de subchaves de 128 bits utilizadas na etapa *AddRoundKey* de cada rodada do algoritmo.

O processo organiza a chave em palavras de 32 bits (W_i) e as expande de forma recursiva através da fórmula $W_i = W_{i-n_k} \oplus Temp$, onde a variável Temp assume o valor da palavra anterior W_{i-1} ou, no caso de limites de bloco (i múltiplo de N_k), passa pela função não linear g().

Essa função g() aplica sequencialmente uma rotação cíclica de bytes (**RotWord**), uma substituição não linear via tabela S-Box (**SubWord**) e uma operação XOR com uma constante de rodada baseada no Corpo Finito de Galois (**Rcon**), garantindo o efeito avalanche, a eliminação de simetrias e a resistência contra ataques de chaves relacionadas.



4.1. Especificações das Variantes

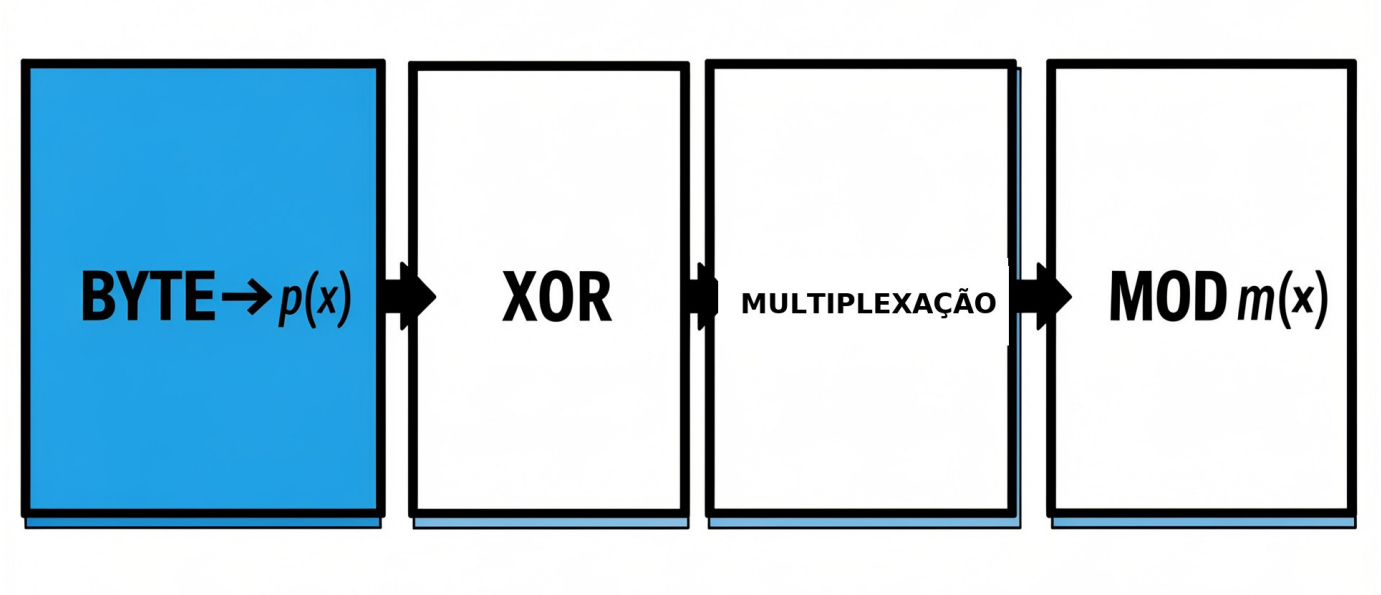
Independentemente do tamanho da chave, o **tamanho do bloco é sempre fixo em 128 bits** (16 bytes), organizados internamente como uma matriz de 4 × 4 bytes denominada **Estado** (*State*).

Variante do Algoritmo	Tamanho da Chave (Key Size)	Ciclos de Transformação (Rounds)	Nível de Segurança e Caso de Uso
AES-128	128 bits (16 bytes)	10 rodadas	Segurança padrão de alta performance para uso corporativo e comercial.
AES-192	192 bits (24 bytes)	12 rodadas	Segurança elevada para dados governamentais e transações bancárias.
AES-256	256 bits (32 bytes)	14 rodadas	Segurança máxima ("grau militar"), resistente a ataques por força bruta atuais e futuros (incluindo computação quântica).

5. Fundamentação Matemática em Corpos Finitos

Toda a eficiência e resistência analítica do AES se fundamentam no campo da álgebra abstrata, utilizando cálculos operados em **Corpos Finitos de Galois** (*Galois Fields*), especificamente denotados por **GF(2⁸)**.

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (\text{ou } 0\text{x}11\text{B em hexadecimal})$$



- **BYTE → p(x):** representa o byte como um polinômio binário, com cada bit como coeficiente.
- **XOR:** é a adição no campo GF(2⁸), feita bit a bit sem transporte.
- **MULTIPLEXAÇÃO:** combina os polinômios para realizar a operação no campo.
- **MOD m(x):** reduz o resultado pelo polinômio irredutível m(x), mantendo-o dentro de GF(2⁸).

- Os bytes do bloco são tratados como polinômios de grau menor que 8 com coeficientes binários.
- A adição corresponde à operação XOR (⊕).
- A multiplicação é realizada módulo o polinômio irredutível: m(x) = x⁸ + x⁴ + x³ + x + 1 (ou 0x11B em hexadecimal)

GF(2⁸)	polinômios de grau menor que 8
byte	
XOR (⊕)	m(x) = x⁸ + x⁴ + x³ + x + 1